

中国科学院大学

《计算机组成原理(研讨课)》实验报告

姓名 黄静怡 学号 2024K8009907042 专业 计算机科学与技术
实验项目编号 4.3 实验名称 深度学习算法与硬件加速器

- 注 1: 撰写此 Word 格式实验报告后以 PDF 格式保存 SERVE CloudIDE 的 `/home/serve-ide/cod-lab/reports` 目录下 (注意: reports 全部小写)。文件命名规则: `prjN.pdf`, 其中 `prj` 和后缀名 `pdf` 为小写, `N` 为 1 至 4 的阿拉伯数字。例如: `prj1.pdf`。PDF 文件大小应控制在 5MB 以内。此外, 实验项目 5 包含多个选做内容, 每个选做实验应提交各自的实验报告文件, 文件命名规则: `prj5-projectname.pdf`, 其中 “-” 为英文标点符号的短横线。文件命名举例: `prj5-dma.pdf`。具体要求详见实验项目 5 讲义。
- 注 2: 使用 `git add` 及 `git commit` 命令将实验报告 PDF 文件添加到本地仓库 master 分支, 并通过 `git push` 推送到实验课 SERVE GitLab 远程仓库 master 分支 (具体命令详见实验报告)。
- 注 3: 实验报告模板下列条目仅供参考, 可包含但不限定如下内容。实验报告中无需重复描述讲义中的实验流程。

完整实验流程包括设计、开发、调试三个步骤:

- **设计**步骤为在代码编写前的准备阶段, 主要包括模块划分和模块结构设计, 并绘制设计图纸。
- **开发**步骤为在 S-IDE 集成开发环境中进行 Verilog/C 代码编写, 并实现完整功能。
- **调试**步骤包括本地仿真验证后推送至云平台进行 FPGA 原型验证, 以及后续迭代调试的全过程。

一、设计成果(对设计流程的总结。需附上设计结构图, 可用 PPT 或者在线绘图软件绘制逻辑结构框图后保存为 PDF, 再插入到 L^AT_EX 中)

```
for(no = 0; no < Oc; no++) → 输出特征图数量
{
    for(ni = 0; ni < Ic; ni++) → 输入特征图数量
    {
        for(y = 0; y < Oh; y++)
        for(x = 0; x < Ow; x++) } 输出特征图尺寸
        {
            if (ni == 0)
                output_feature_map(no, x, y) = bias(no);
            for(ky = 0; ky < K; ky++)
            for(kx = 0; kx < K; kx++) } 权重值尺寸
            {
                iw = kx + x * S;
                ih = ky + y * S;
                output_feature_map(no, x, y) +=
                    input(ni, iw, ih) * weight(ni, no, kx, ky);
            }
        }
    }
}
```

图 1: 卷积算法伪代码

二、 开发成果(对代码开发流程的总结。需附上包含注释的 VerilogHDL 关键代码段、相应信号的仿真波形和信号变化的说明)

1 conv.c 中 void convolutional() 函数的实现

convolutional 函数用于完成卷积操作。

对中间变量的定义如下：

```
//由于所有的输入,kernel和输出都是在内存中连续的排列，所以直接用线性地址
unsigned input_size = mul(input_fm_h,input_fm_w);    //输入的大小
unsigned filter_size = 1 + mul(weight_size.d2, weight_size.d3); //每组kernel的大小，
    每个kernel的第一个元素是bias
    //每个kernel都有一个输出

unsigned num_out; //输出个数
unsigned num_in;  //输入个数
unsigned bias;

short input_address;
short filter_outer_address;

//一些即将用到的的线性坐标
short filter_line_addr;
short input_line_addr;

//x*S, y*S
short x_stride;
short y_stride;

//用于标记输出相对于基址的偏移
short position = 0;

//输入的位置
short iw;
short ih;

//用于暂存结果，用int来防止溢出
int value_temp;
short value_true;

for(num_out = 0; num_out < conv_size.d1; num_out++){
    filter_outer_address = mul(num_out,filter_size); //调整到对应的filter
    bias = weight[filter_outer_address];
    //当前输出所在的位置
    for(int y = 0; y < conv_out_h ; y++){
        y_stride = mul(y,stride);
        for(int x = 0; x < conv_out_w; x++){
            value_temp = 0;
            x_stride = mul(x,stride);
```

```

for(num_in = 0; num_in < rd_size.d1; num_in++){
    input_address = mul(num_in,input_size);
    for(int ky = 0; ky < weight_size.d2; ky++){
        ih = ky + y_stride - pad; //找到对应的输入的位置，放在外面进行可以减少乘法的计算次数，
        //要考虑pad
        if(ih < 0 || ih >= input_fm_h){
            continue; //考虑pad的情况，pad在最周围一圈，放在之前来减少乘法的计算次数
        }
        filter_line_addr = mul(ky,weight_size.d3);//注意是宽度
        input_line_addr = mul(ih,input_fm_w);
        for(int kx = 0; kx < weight_size.d3; kx++){
            iw = kx + x_stride - pad;
            if(iw < 0 || iw >= input_fm_w){
                continue;
            }
            //weight里的i用于跳过bias
            value_temp = value_temp + mul(in[input_address + input_line_addr + iw], weight[1
                + filter_outer_address + filter_line_addr + kx]);
        }//kx
    }//ky
}//in
value_true = (value_temp >> FRAC_BIT);
out[position] = (short)(bias + value_true);
position++; //计算之后的值
//out[position] = (short)(bias + value_temp >> FRAC_BIT);
}//x
}//y
}//no
}

```

主要就是利用了线性存储的行优先存储的特征，因为是行优先，所以对于 (i,j) 位置的元素，其线性坐标为 $\text{line_addr} = i \times \text{width} + j$ ，于是我们可以根据而借此来索引输入和 filter 中的元素，

另外需要注意的是，由于 filter 的第一个元素是 bias，所以在计算时需要将其单独处理，另外值得一提的是在这次实验中我们考虑了边界 padding 的情况，具体实现就是根据当前的坐标计算出目前的 ih 和 iw，并且根据 ih 和 iw 的值是否超出输入的范围来判断，另外我将对于行的判断放在了倒数第二个循环，来尽可能的减少循环次数

2 conv.c 中 void pooling() 函数的实现

池化的代码的思路和卷积部分基本上类似，有关 padding 的部分是完全相同的。依然是利用线性存储的特点来对数据进行索引，与卷积不同的地方在于，在每次设置的过程中，维护一个 max 值来存储每一次的最大值，另外需要注意的是，在池化的过程中，需要对输入的大小进行判断，如果输入的大小小于 pool_size，则直接将输入作为输出这个 max 在每一个元素被赋值前初始化位最小的 short 型，并之后每次与当前的值进行比，取最大值，这样就可以得到池化的结果。

3 conv.c 中 void launch_hw_accel() 函数的实现

```
*gpio_start = 1;
```

```
while(*gpio_done);
*gpio_start = 0;
```

gpio_start 为只写寄存器,其最低位表示加速器启动。因此,向 START 的第 0 位写 1,启动加速器。

gpio_done 为只读寄存器,其最低位表示加速器工作状态。因此,不断检测 DONE 寄存器第 0 位是否拉高来判断加速器是否已完成卷积操作。当加速器完成卷积操作后,向 START 的第 0 位写 0,停止加速器。

4 costum_cpu.v 中对 mul 指令的实现

```
//MUL
assign MULResult = RF_rdata1 * RF_rdata2;

//Result
always @(posedge clk) begin
    if (current_state[isEX]) begin
        Result <= {(32){ALUEn}} & ALUResult
            | {(32){ShiftEn}} & ShifterResult
            | {(32){MULEn}} & MULResult;
    end
end
```

引入了 MULEn 控制信号,来从一系列运算数据中选择结果数据。其中 ALUEn,ShiftEn,MULEn 三条控制信号各自互斥,定义如下

```
assign ALUEn = (OP & ~funct7[0] | OP_IMM) & (funct3[1] | ~funct3[0]) | JALR | LOAD | S_Type |
    B_Type;
assign ShiftEn = (OP & ~funct7[0] | OP_IMM) & (~funct3[1] & funct3[0]);
assign MULEn = OP & funct7[0];
```

5 conv.c 中 int main() 函数内性能计数器的实现

```
bench_done(&res);
printf("====Hardware Performance Counter====\n");
printf("Cycle Count:          %u\n", res.cnt[0]);
printf("Instruction Count:      %u\n", res.cnt[1]);
printf("Memory Read:             %u\n", res.cnt[2]);
printf("Memory Write:            %u\n", res.cnt[3]);
printf("Instruction Request Delay: %u\n", res.cnt[4]);
printf("Instruction Response Delay: %u\n", res.cnt[5]);
printf("MemRead Request Delay: %u\n", res.cnt[6]);
printf("Read Data Delay:          %u\n", res.cnt[7]);
printf("MemWrite Request Delay: %u\n", res.cnt[8]);
printf("Branch Count:             %u\n", res.cnt[9]);
printf("Jump Count:               %u\n", res.cnt[10]);
printf("=====\n");
```

先在卷积与池化前定义 Result 类型变量,用于存储性能计数,并运行 bench_prepare;然后当卷积、池化、比对结束时,运行 bench_done,并输出相应结果。各个性能计数器的定义同之前几次实验,不再赘述。

三、 学习总结(总结设计、开发与调试过程中遇到的问题,及实验中的收获与不足)

1 问题总结(实验过程中遇到的问题、对问题的思考过程及解决方法)

在最初的卷积算法实现时,我先对每组乘数与被乘数相乘并移位(使得结果仍然为 16-bit 定点数),然后将所有结果相加。然而,这样其实是存在问题的。事实上,这样得到的运行结果与标准结果存在微小的差异:尽管大部分答案正确,但仍存在少部分数据与标准答案相差一二。经过反复比对,我发现先移位后相加的操作会丢失原本移位前低位的数据,带来精度上的损失。这些精度上的损失表现为结果与标准略有偏差。解决方案是将先移位后相加改成先相加后移位。

最初我将性能计数器放在了 hit_good_trap 函数之后,导致计数结果无法正常显示。解决方案是将其放在 hit_good_trap 判定之前。至于导致这个问题的原因,我猜测是因为该程序在 hit_trap 时就已经退出了,所以并不会执行接下来的代码。

序号	问题类别	出错模块	问题现象	产生原因	解决办法
1	精度损失	卷积算法	运算优先级	操作顺序	修改顺序
2	性能计数器	main	无法正常打印	顺序问题	修改顺序

表 1: 实验问题总结

2 收获总结(在数字电路设计方法、RTL 编写方法、数字电路验证方法及其他方面的收获)

本次实验比较简单,难点在于对于池化和卷积这两个算法及其具体实现的理解,另外就是对于 padding 的处理,由于 padding 的处理需要考虑到边界的情况,所以需要对输入的坐标进行判断,如何判断才能正确的处理边界情况,怎样的判断才能尽可能地减少时间开销也是本次实验需要考虑的

类别	名称	体会	后续计划
对 cpu 的理解	理解多周期 cpu	更加熟悉 cpu 运行原理,即几个阶段的整合	多复习讲义
代码编写	编写 verilog 并实现 always 逻辑	代码编写更加熟练	编写代码
debug 能力	寻找错误	寻找代码错误原因更加熟练	辅助工具实现 debug

表 2: 收获总结

四、 扩展思考(对讲义中思考题(如有)的理解和回答)

● 卷积中如果使用边界填充,算法应如何修改?

对 ih 和 iw 变量设置了一定的偏移,使得它们的枚举起点考虑了因边界存在而带来的偏移。注意到边界填充的数据均为零,且不存在实际内存中。所以当两个访问坐标 iw 和 ih 有一个超出原特征图边界时,将直接跳过这一访问。其合理性在于,值为零的样本点不会对和造成影响。

● 在软件算法实现中,需考虑如何避免出现溢出和精度损失

在问题总结 2 中

五、 实验时长

在课后,你花费了大约 7 小时完成此次实验,其中各个阶段的用时如表 3 所示。

步骤	时长(小时)	说明
设计	2	认真学习课程内容,理解所需实现的板块
开发	2	完成实验代码的编写
调试	3	进行测试样例的测试,不断修正错误

表 3: 实验时长